

BreezeML: a production-aware machine learning workflow layer with statistical honesty by default

Akash Anipakalu Giridhar ¹

¹ Independent Researcher, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 18 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

BreezeML is a Python library that wraps the scikit-learn ecosystem (Pedregosa et al., 2011) in a workflow layer designed around one principle: the statistically sound path should be the shortest path. A complete, leakage-safe, stratified, seeded classification workflow is three lines:

```
from breezeml import datasets, fit, predict

model = fit(datasets.iris(), "species")
preds = predict(model, new_df)
```

Behind that call, BreezeML applies leakage-safe preprocessing inside cross-validation folds, stratified splitting, fixed random seeds, and evaluation against a naive baseline. The returned Model object carries the full lifecycle: `evaluate()`, `explain()`, `card()` for an automatically caveated model card (Mitchell et al., 2019), `conformal predict_interval()` and `predict_set()` with distribution-free coverage guarantees (Angelopoulos & Bates, 2023; Vovk et al., 2005), `export()` for a standalone scikit-learn script, and `deploy()` for a FastAPI service with drift monitoring.

The library spans the applied workflow beyond core training: automated model comparison and tuning (optionally via Optuna (Akiba et al., 2019)), automatic feature engineering with out-of-fold target encoding, imbalance handling with threshold tuning and calibration, permutation importance and partial dependence, fairness reports with four-fifths rule verdicts (Feldman et al., 2015), statistical significance tests for model comparison (McNemar, paired cross-validation t-test) (Dietterich, 1998), time series forecasting with walk-forward validation, survival analysis, semi-supervised and active learning, anomaly detection, causal effect estimation, and lightweight experiment tracking.

Statement of need

Data leakage and missing baselines are a documented, field-wide reproducibility problem: Kapoor & Narayanan (2023) catalogue leakage-driven errors in 294 papers across 17 fields, and Kaufman et al. (2012) show how easily leakage arises in routine preprocessing. The users most exposed to these pitfalls, students, analysts, and increasingly autonomous AI agents, are the same users least equipped to detect them. Existing low-code libraries reduce boilerplate but not risk: they will happily fit preprocessing on the full dataset, report accuracy without a baseline, and ship an imbalanced classifier without a warning.

BreezeML's contribution is a design that makes methodological rigor the default rather than an option. Its target audiences are educators and students who need sound defaults while learning; analysts who need to go from a DataFrame to an audited, deployable model quickly; and researchers building agent systems who need ML tooling that resists silent statistical misuse.

State of the field

Low-code ML layers over scikit-learn are well established. PyCaret (Ali, 2020) offers a broad experiment-management workflow; LazyPredict (Pandala, 2022) fits many models quickly for triage; auto-sklearn (Feurer et al., 2015) automates model and hyperparameter search. BreezeML differs on three axes rather than competing on breadth of estimators:

1. **Statistical guardrails as defaults.** None of the above evaluates every model against a mandatory naive baseline, audits for single-feature target leakage, or refuses naive regression on censored survival data. BreezeML does all three out of the box.
2. **Footprint.** The core installs with exactly four dependencies (scikit-learn, pandas (McKinney, 2010), NumPy (Harris et al., 2020), joblib), enforced by a CI test that fails if the contract is violated. On identical hardware, a fresh BreezeML install is 274 MB in under three minutes versus 952 MB in over six minutes for PyCaret, with a 3.1 s cold import versus 6.9 s (methodology and caveats in the repository's benchmarks/ and docs/benchmarks.md).
3. **Exit path.** `export()` writes a standalone scikit-learn training script reproducing the exact fitted pipeline with no BreezeML import. Contributing these guardrails upstream to any single library was not viable because they span the whole workflow (training, comparison, deployment, monitoring) and depend on an opinionated, unified API surface; hence a new package.

Software design

Three design decisions define the library:

An honest verdict as a first-class object. `report(model, df)` runs cross-validated performance against a naive baseline, a leakage audit, class imbalance checks, and optional fairness analysis, and reduces them to a single SHIP / WARN / STOP verdict, serializable to JSON or Markdown. A model that fails to beat the naive baseline, or shows target leakage, is flagged before it is deployed.

AI agents as first-class users. A built-in Model Context Protocol (MCP) server exposes training, comparison, evaluation, and deployment as agent tools, and exposes the same SHIP / WARN / STOP verdict, so a language-model agent is expected to confirm SHIP before calling `deploy()`. To our knowledge no other ML workflow library ships this guardrail pattern for agent-driven ML.

Teach, then get out of the way. `explain_decisions=True` narrates every preprocessing and modeling choice in plain language as it happens, and the zero lock-in export means a student who starts with three lines graduates to reading, and eventually writing, the equivalent scikit-learn code. The main trade-off accepted throughout is implementing capabilities (significance tests, conformal wrappers, drift metrics) on the four core dependencies alone rather than importing specialized packages, trading some sophistication for a dependency contract that keeps the library viable in classrooms, constrained environments, and agent sandboxes.

The package is typed (PEP 561), tested in CI by a suite spanning 29 test modules, and versioned under a documented semantic-versioning and deprecation policy.

Research impact statement

BreezeML is early-stage but in active use: it is distributed on PyPI, where it records on the order of two thousand downloads per month excluding mirrors (per pypistats.org), and is documented at breezempl.org with an in-browser playground. Its near-term research relevance is twofold. For ML education and applied analysis, it operationalizes published recommendations on leakage avoidance (Kapoor & Narayanan, 2023) and transparent model reporting (Mitchell

et al., 2019) as executable defaults rather than guidelines. For research on autonomous AI agents, its MCP server plus machine-readable SHIP / WARN / STOP verdict provides a concrete, inspectable mechanism for constraining agent-driven model training and deployment, a pattern we expect to be of increasing interest as agent systems take on data-science tasks.

AI usage disclosure

BreezeML was designed and developed by the author, who conceived the library, its architecture, its API, its statistical-honesty design principles, and its benchmark methodology. Generative AI (Claude, Anthropic; used via the Claude Code development environment) was used as a development assistant: helping implement portions of the codebase, documentation, and tests under the author's direction, and assisting with drafting and editing this manuscript. All AI-assisted content was reviewed, modified where needed, and validated by the author, who takes full responsibility for the accuracy, originality, and licensing compliance of all submitted materials.

Acknowledgements

BreezeML builds on the scientific Python ecosystem, in particular scikit-learn, pandas, NumPy, and joblib.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- Ali, M. (2020). *PyCaret: An open source, low-code machine learning library in Python*. <https://pycaret.org>
- Angelopoulos, A. N., & Bates, S. (2023). Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4), 494–591. <https://doi.org/10.1561/2200000101>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- Feldman, M., Friedler, S. A., Moeller, J., Scheidegger, C., & Venkatasubramanian, S. (2015). Certifying and removing disparate impact. *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 259–268. <https://doi.org/10.1145/2783258.2783311>
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J. T., Blum, M., & Hutter, F. (2015). Efficient and robust automated machine learning. *Advances in Neural Information Processing Systems* 28, 2962–2970.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. <https://doi.org/10.1016/j.patter.2023.100804>

- 125 Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining:
126 Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from*
127 *Data*, 6(4), 1–21. <https://doi.org/10.1145/2382577.2382579>
- 128 McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the*
129 *9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- 130 Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer,
131 E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings*
132 *of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 220–229.
133 <https://doi.org/10.1145/3287560.3287596>
- 134 Pandala, S. R. (2022). *LazyPredict: Build many models with little code*. [https://github.com/](https://github.com/shankarpandala/lazypredict)
135 [shankarpandala/lazypredict](https://github.com/shankarpandala/lazypredict)
- 136 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
137 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
138 Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python.
139 *Journal of Machine Learning Research*, 12, 2825–2830.
- 140 Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic learning in a random world*.
141 Springer. <https://doi.org/10.1007/b106715>